Primer caso de estudio
Estructuras de Datos

Estructuras de Datos
Universidad Cenfotec

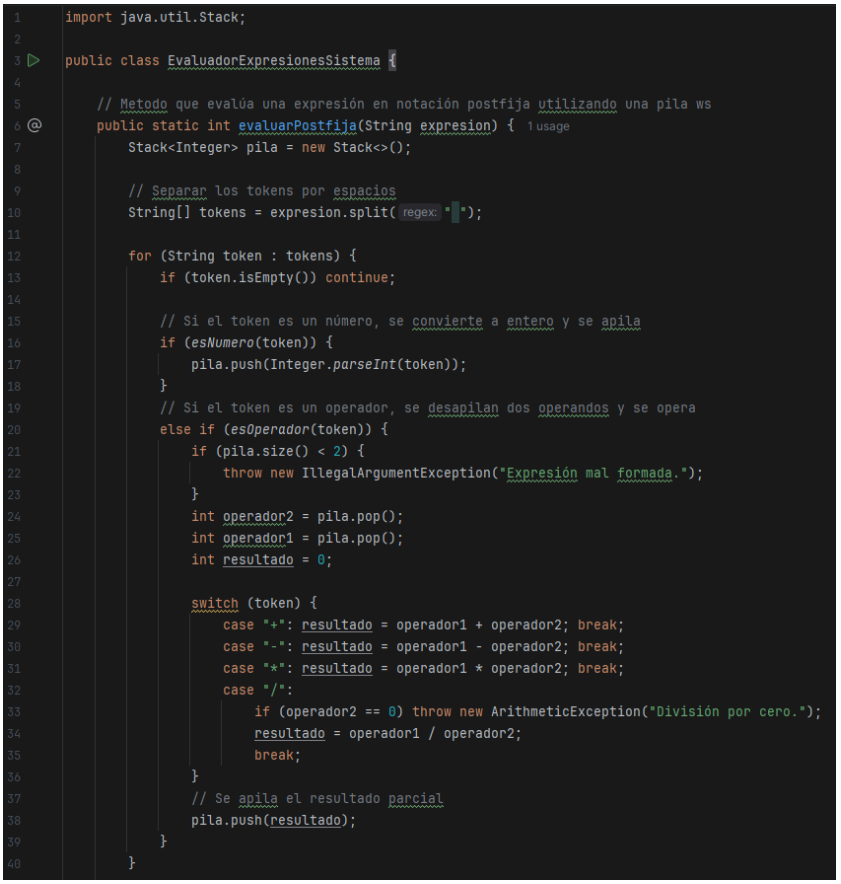
Estudiante: Luis Alonso Agüero Suárez
Profesor: Romario Salas Cerdas

Explicación: ¿Por qué una pila es la estructura de datos adecuada?
La elección de una pila como estructura de datos para el análisis de expresiones aritméticas se fundamenta en los requerimientos lógicos y formales de la matemática y el procesamiento de lenguajes. En primer lugar, las expresiones aritméticas involucran un requerimiento crítico de anidamiento y control de signos de agrupación (como paréntesis, corchetes y llaves), donde el último elemento de apertura introducido debe ser el primero en cerrarse y validarse (comportamiento LIFO). En segundo lugar, responde al requerimiento de jerarquía, precedencia y asociatividad de los operadores, permitiendo diferir el cálculo de operaciones de menor prioridad mientras se evalúan aquellas de mayor jerarquía. Todo esto se complementa de forma óptima con la disciplina de orden estricto, la seguridad en el control de acceso a los tokens desde la cima y la eficiencia algorítmica de orden constante O(1) que caracteriza a las operaciones fundamentales de una pila, convirtiéndola en el mecanismo idóneo para el análisis sintáctico en software de sistema.

Comentario:
La utilización de la pila aporta una seguridad robusta al restringir el acceso a los datos exclusivamente a su cima, lo que previene alteraciones indebidas de los tokens y detecta tempranamente errores sintácticos como paréntesis desbalanceados. En cuanto al orden, establece una disciplina secuencial y determinista que respeta fielmente la jerarquía matemática de las operaciones. Finalmente, garantiza una alta eficiencia gracias a que sus operaciones fundamentales operan con una complejidad temporal de orden constante $O(1)$, optimizando el rendimiento y el uso de recursos en el software de sistema.

Descripción de un ejemplo concreto:

Un ejemplo claro de esta funcionalidad en el software de sistema se encuentra en los intérpretes de comandos y compiladores (como la Máquina Virtual de Java - JVM o intérpretes de lenguajes integrados en sistemas operativos), los cuales utilizan pilas internamente para analizar, validar y evaluar expresiones aritméticas escritas por el usuario. Cuando ejecutas una operación matemática compleja en una terminal o un script del sistema, el analizador léxico y sintáctico descompone la expresión en tokens y emplea una estructura de pila para evaluar las operaciones respetando rigurosamente la jerarquía matemática y el uso de paréntesis en tiempo de ejecución.



```java
import java.util.Stack;

public class EvaluadorExpresionesSistema {

    // Metodo que evalúa una expresión en notación postfija utilizando una pila ws
    public static int evaluarPostfija(String expresion) {  1 usage
        Stack<Integer> pila = new Stack<>();

        // Separar los tokens por espacios
        String[] tokens = expresion.split( regex: " ");

        for (String token : tokens) {
            if (token.isEmpty()) continue;

            // Si el token es un número, se convierte a entero y se apila
            if (esNumero(token)) {
                pila.push(Integer.parseInt(token));
            }
            // Si el token es un operador, se desapilan dos operandos y se opera
            else if (esOperador(token)) {
                if (pila.size() < 2) {
                    throw new IllegalArgumentException("Expresión mal formada.");
                }
                int operador2 = pila.pop();
                int operador1 = pila.pop();
                int resultado = 0;

                switch (token) {
                    case "+": resultado = operador1 + operador2; break;
                    case "-": resultado = operador1 - operador2; break;
                    case "*": resultado = operador1 * operador2; break;
                    case "/":
                        if (operador2 == 0) throw new ArithmeticException("División por cero.");
                        resultado = operador1 / operador2;
                        break;
                }
                // Se apila el resultado parcial
                pila.push(resultado);
            }
        }
```



```
        if (pila.size() != 1) {
            throw new IllegalArgumentException("Expresión inválida.");
        }

        return pila.pop();
    }

    private static boolean esNumero(String token) {  1 usage
        try {
            Integer.parseInt(token);
            return true;
        } catch (NumberFormatException e) {
            return false;
        }
    }

    private static boolean esOperador(String token) {  1 usage
        return token.equals("+") || token.equals("-") || token.equals("*") || token.equals("/");
    }

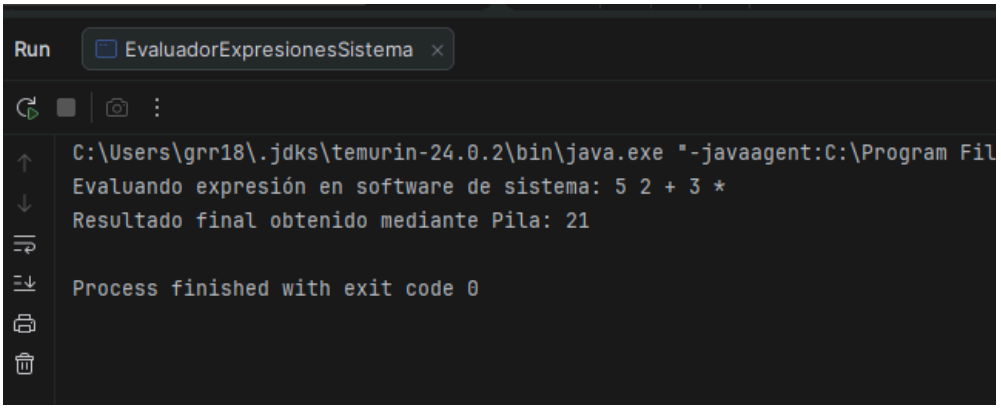
    public static void main(String[] args) {
        // Ejemplo de expresión postfija equivalente a: (5 + 2) * 3
        String expresionEjemplo = "5 2 + 3 *";

        System.out.println("Evaluando expresión en software de sistema: " + expresionEjemplo);
        int resultado = evaluarPostfija(expresionEjemplo);
        System.out.println("Resultado final obtenido mediante Pila: " + resultado);
    }
}
```

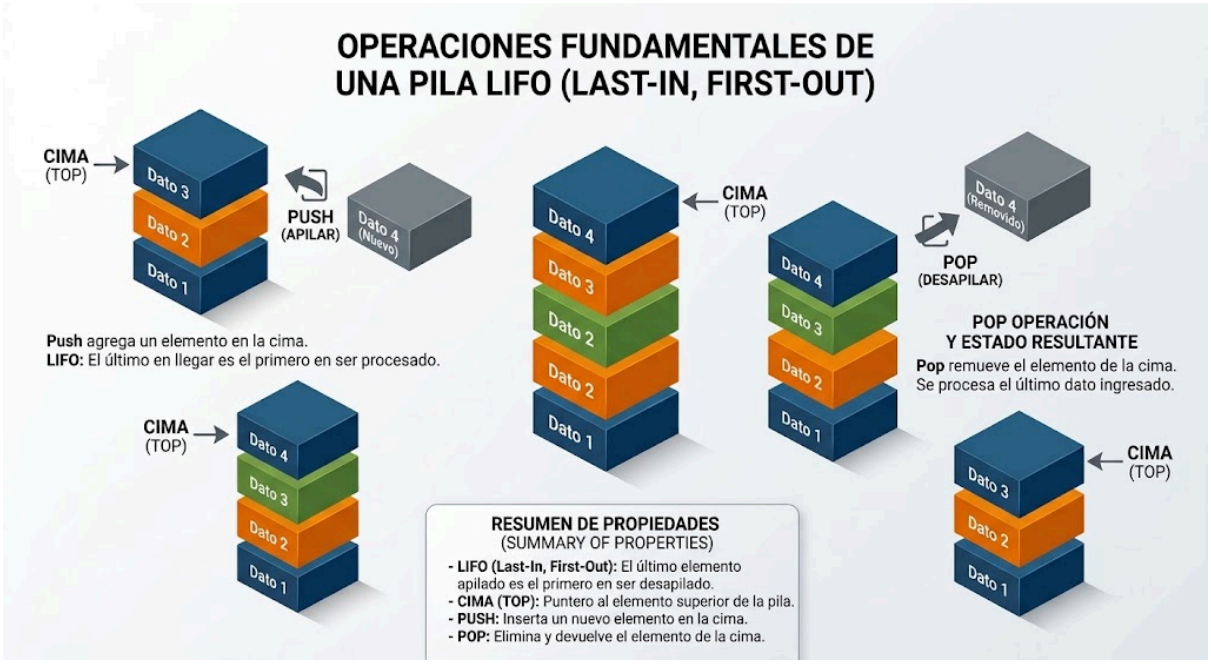
Explicación del comportamiento del código: El programa implementado simula el funcionamiento de un evaluador de expresiones en un software de sistema mediante el uso de una pila. La lógica opera bajo el siguiente flujo:

1. Lectura y separación de tokens: La expresión de entrada se divide en fragmentos individuales o tokens (números y operadores) utilizando espacios como delimitadores.
2. Almacenamiento de operandos (Push): A medida que el bucle recorre los tokens, si encuentra un número literal, lo convierte a entero y lo almacena temporalmente en la pila.
3. Procesamiento de operadores y comportamiento LIFO (Pop): Al detectar un operador aritmético (+, -, *, /), el sistema extrae automáticamente de la pila los dos últimos números ingresados. Esto refleja el comportamiento LIFO (Last In, First Out), asegurando que los operandos más recientes sean los primeros en operarse.
4. Resultado parcial: Una vez efectuada la operación matemática correspondiente, el resultado se vuelve a apilar en la estructura,

permitiendo evaluar expresiones complejas de manera eficiente y respetando la jerarquía de las operaciones en tiempo de ejecución.

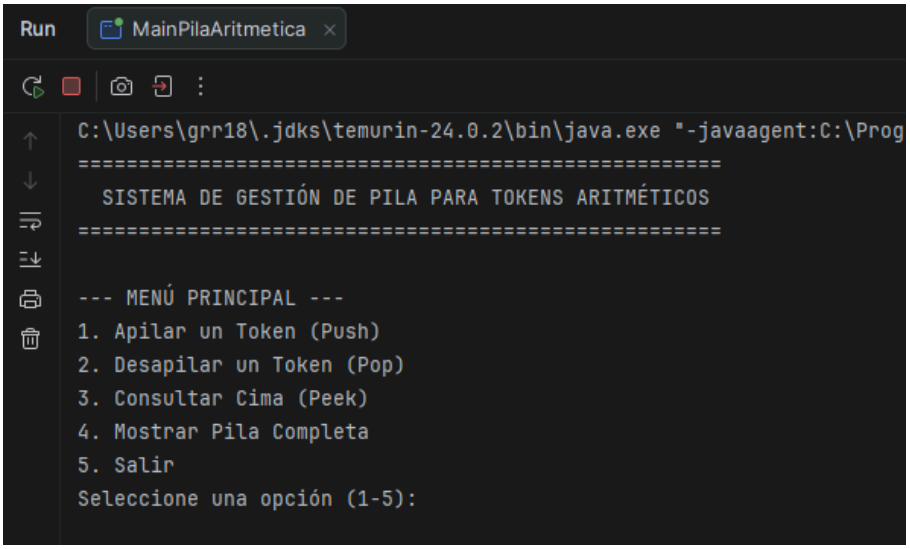


Representación gráfica de las operaciones fundamentales de la pila:



Pila Explicada:

Figura. Ejecución del menú interactivo en la consola: Muestra la interfaz de usuario en tiempo real del sistema de gestión de pila para tokens aritméticos, permitiendo elegir entre las opciones de apilar, desapilar, consultar la cima y visualizar la estructura de datos.



Código Completo para ejecutar:

```java
import java.util.Scanner;

public class MainPilaAritmetica {
   public static void main(String[] args) {
       Scanner scanner = new Scanner(System.in);
       PilaTokens pila = new PilaTokens();
       int opcion = 0;

       System.out.println("==========");
       System.out.println("  SISTEMA DE GESTIÓN DE PILA PARA TOKENS ARITMÉTICOS ");
       System.out.println("==========");

       do {
           System.out.println("\n--- MENÚ PRINCIPAL ---");
           System.out.println("1. Apilar un Token (Push)");
           System.out.println("2. Desapilar un Token (Pop)");
           System.out.println("3. Consultar Cima (Peek)");
           System.out.println("4. Mostrar Pila Completa");
           System.out.println("5. Salir");
           System.out.print("Seleccione una opción (1-5): ");

           if (scanner.hasNextInt()) {
               opcion = scanner.nextInt();
               scanner.nextLine(); // Limpiar el buffer de entrada

               switch (opcion) {
                   case 1:
                       System.out.print("Ingrese el token aritmético (ej: 5, x, +, (, etc.): ");
                       String token = scanner.nextLine();
                       if (!token.trim().isEmpty()) {
                           pila.push(token);
                       } else {
                           System.out.println("Error: El token no puede estar vacío.");
                       }
                       break;

                   case 2:
                       pila.pop();
                       break;

                   case 3:
                       String cima = pila.peek();
                       if (cima != null) {
                           System.out.println("\n-> El token actual en la CIMA es: " + cima);
                       }
                       break;

                   case 4:
                       pila.mostrarPila();
                       break;
```

```
                case 5:
                    System.out.println("\n¡Gracias por utilizar el sistema de pilas! Saliendo...");
                    break;

                default:
                    System.out.println("Opción inválida. Por favor, seleccione un número entre 1 y 5.");
            }
        } else {
            System.out.println("Error: Debe ingresar un valor numérico válido.");
            scanner.next(); // Descartar entrada incorrecta
        }

    } while (opcion != 5);

    scanner.close();
  }
}
```